

3D Object Detection

Literature Survey

Vathsai Upadhyay
B.Tech III Year
June 2026

Introduction

- Three-dimensional object detection is one of the most active research areas in computer vision today. Unlike 2D detection, which operates on flat images and outputs rectangular boxes, 3D detection operates on point clouds — unordered sets of (x, y, z) coordinates captured by LIDAR or depth cameras — and outputs oriented 3D bounding boxes with full spatial extent. For each detected object, the model must predict seven parameters: center coordinates (x, y, z) , physical dimensions (length, width, height), and heading angle (yaw). The irregular, sparse nature of point cloud data makes this significantly harder than its 2D counterpart.
- The field is driven by two high-stakes domains. In **indoor scene understanding**, the goal is to identify furniture and everyday objects within a single room using an RGB-D sensor. Standard benchmarks are ScanNet V2 (1,513 annotated 3D room scans, 18 object classes) and SUN RGB-D (10,355 RGB-D images, 10 categories). In **outdoor autonomous driving**, the goal is to detect vehicles, pedestrians, and cyclists in street-scale LIDAR scans in real time, with KITTI, nuScenes, and the Waymo Open Dataset serving as the primary benchmarks.
- The 2018–2021 period surveyed here saw 3D detection transition from hand-crafted feature engineering to fully learned, end-to-end systems. Three architectural families emerged: **point-based methods** (VoteNet, F-PointNet, PointRCNN, ImVoteNet) process raw points directly using PointNet++ as a backbone; **voxel-based methods** (VoxelNet, PointPillars) discretize the cloud onto a regular grid, enabling standard convolutions at the cost of quantization; and **transformer-based methods** (3DETR, GroupFree-3D) apply attention across point features to capture global context without hand-tuned locality constraints. Hybrid architectures — PV-RCNN and CenterPoint — combine multiple families for complementary gains.
- The standard evaluation metric is **mean Average Precision (mAP)** at a fixed IoU threshold between predicted and ground-truth boxes. Indoor methods report mAP at $\text{IoU} \geq 0.25$ (loose) and $\text{IoU} \geq 0.50$ (strict). Outdoor methods on KITTI report 3D AP across Easy, Moderate, and Hard difficulty tiers, with Moderate as the primary comparison point.

1. VoteNet (CVPR 2019)

Paper: <https://arxiv.org/abs/1904.09664>

Code: <https://github.com/facebookresearch/votenet>

Citation: Qi, C. R., Litany, O., He, K., & Guibas, L. J. (2019). Deep Hough Voting for 3D Object Detection in Point Clouds. CVPR 2019.

Abstract

VoteNet is the first major end-to-end neural network built specifically for detecting 3D objects in indoor scenes — working directly on raw point clouds without any intermediate conversion to images or voxels. The core problem it tackles is surprisingly subtle: the geometric center of most objects (e.g., the center of a chair) lies in empty space, not on any surface point. This makes it impossible for a network to directly "see" and predict object centers from observed points.

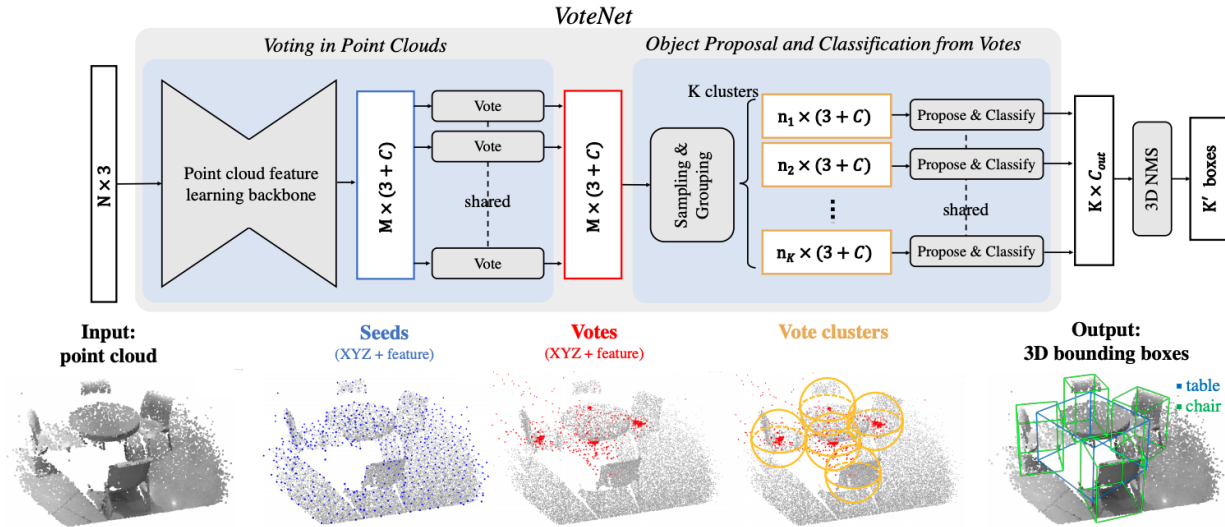
VoteNet solves this by borrowing the classical Hough voting idea from computer vision: every surface point independently predicts a small offset vector pointing toward its parent object's center. These predicted "vote points" naturally cluster around true object centers. The clusters are then grouped and classified to produce final 3D bounding box predictions. The entire pipeline — backbone, voting module, and detection head — is trained end-to-end using a combined loss that supervises both the vote offsets and the final box parameters.

The backbone is PointNet++, which hierarchically aggregates local neighborhoods of points into increasingly abstract feature representations. VoteNet demonstrated for the first time that a pure point-cloud 3D detector could match and exceed methods that relied on RGB images or hand-crafted features, setting a new standard for indoor 3D detection.

Datasets Used

- ScanNet V2 — 1,513 indoor RGB-D scans across 18 object categories (chairs, tables, sofas, beds, etc.)
- SUN RGB-D — 10,355 RGB-D images across 10 common indoor object categories

Architecture



```

Input point cloud (N = 20,000 points, xyz)
|
v
PointNet++ backbone (Set Abstraction layers)
--> M = 1,024 seed points + feature vectors (C dims each)
|
v
Voting Module (shared MLP per seed point)
--> Each seed predicts (dx, dy, dz) offset toward object center
--> Vote points = seed_xyz + predicted offset
|
v
Vote Aggregation (farthest point sampling + ball query grouping)
--> K = 256 vote clusters (one per candidate object)
|
v
PointNet on each cluster
|
v
Detection Head: Classification + 3D Box Regression (center, size, heading)
  
```

Performance Results

Dataset	Metric	Score
ScanNet V2	mAP @ IoU 0.25	58.6
ScanNet V2	mAP @ IoU 0.50	33.5
SUN RGB-D	mAP @ IoU 0.25	57.7

Open Problems

- Performance degrades substantially at stricter IoU thresholds — mAP drops from 58.6 to 33.5 going from IoU 0.25 to 0.50, indicating imprecise box localization.
 - Voting fails when objects are heavily occluded or consist of scattered, fragmented point clusters — the votes do not cluster cleanly around the true center.
 - No use of RGB color/texture information. All decisions are made purely from 3D geometry, leaving useful semantic cues untapped.
 - Proposal quality is limited by the Euclidean-distance clustering step, which can merge nearby objects or split large ones.
-

2. F-PointNet / Frustum PointNets (CVPR 2018)

Paper: <https://arxiv.org/abs/1711.08488>

Code: <https://github.com/charlesq34/frustum-pointnets>

Citation: Qi, C. R., Liu, W., Wu, C., Su, H., & Guibas, L. J. (2018). Frustum PointNets for 3D Object Detection from RGB-D Data. CVPR 2018.

Abstract

Frustum PointNets (F-PointNet) takes a clever hybrid approach: rather than searching for objects directly in 3D (which is expensive and hard), it uses a mature 2D image detector to first find candidate regions in the image plane, then concentrates all 3D processing only within those regions. This drastically reduces the 3D search space.

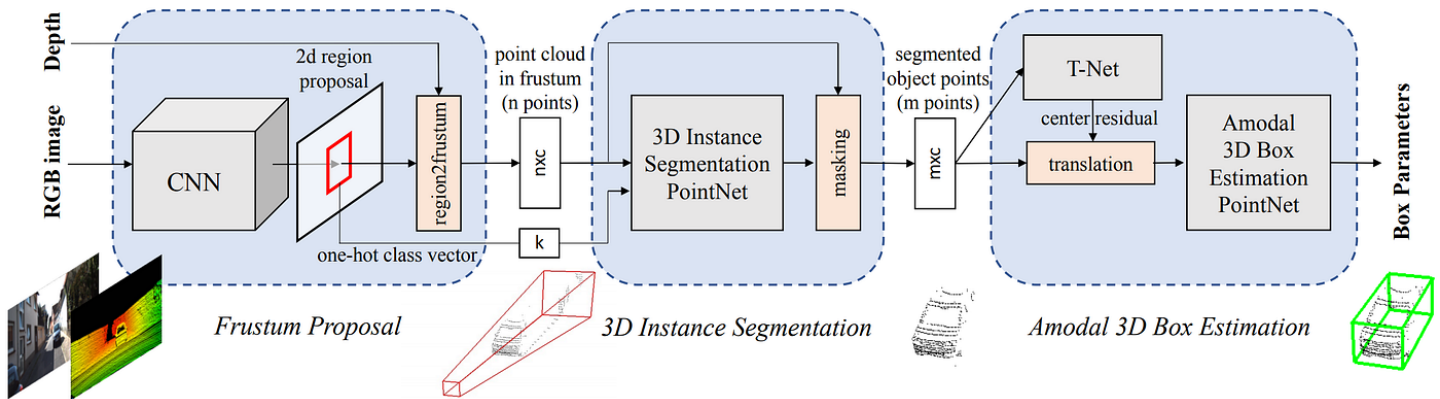
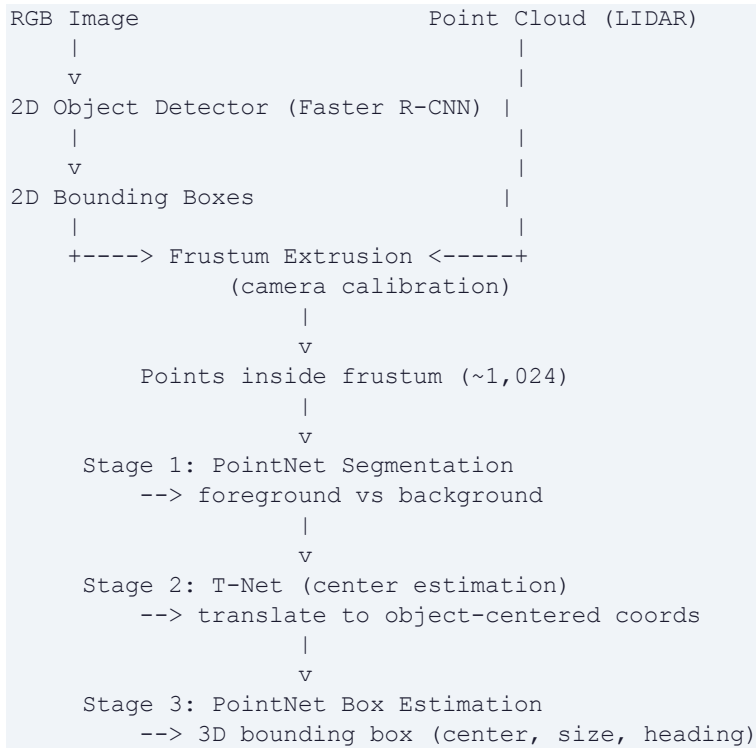
The process begins by running a 2D object detector (e.g., Faster R-CNN) on an RGB image to get 2D bounding boxes. Each box is then "extruded" backward through the camera using known calibration parameters, creating a 3D frustum — a wedge-shaped region of space that must contain the detected object. All LIDAR or depth-camera points falling inside this frustum are extracted.

Within each frustum, a three-stage PointNet pipeline runs: first, an instance segmentation PointNet identifies which points belong to the actual object versus the background; second, a T-Net (Transformation Network) estimates the object's 3D center to normalize the coordinate frame; third, a box estimation PointNet predicts the final 7-parameter 3D bounding box (center x/y/z, dimensions l/w/h, heading angle). At the time of publication, this pipeline achieved state-of-the-art results on the KITTI outdoor benchmark for car detection.

Datasets Used

- KITTI — outdoor autonomous driving scenes with LIDAR + camera. Standard benchmark with Easy/Moderate/Hard difficulty splits per object class (car, pedestrian, cyclist).
- SUN RGB-D — also evaluated on indoor RGB-D scenes using the same frustum pipeline.

Architecture



Performance Results

Dataset	Class	Difficulty	3D AP
KITTI	Car	Easy	81.20
KITTI	Car	Moderate	70.39
KITTI	Car	Hard	62.19
KITTI	Pedestrian	Easy	51.21
KITTI	Cyclist	Easy	71.96

Open Problems

- Detection recall is fundamentally bounded by the 2D detector — any object missed in the image (due to occlusion, small size, unusual viewpoint) can never be recovered in 3D.
 - Frustum overlap is problematic in cluttered scenes: when two objects are close together, their frustums may merge, confusing the segmentation stage.
 - Requires an aligned RGB image alongside the point cloud. Pure LIDAR-only scenarios are not supported without a separate image source.
 - The three-stage sequential pipeline accumulates errors — mistakes in segmentation or T-Net estimation propagate directly into box quality.
-

3. VoxelNet (CVPR 2018)

Paper: <https://arxiv.org/abs/1711.06396>

Code: <https://github.com/qianguih/voxelnet>

Citation: Zhou, Y., & Tuzel, O. (2018). VoxelNet: End-to-End Learning for Point Cloud Based 3D Object Detection. CVPR 2018.

Abstract

VoxelNet was a landmark contribution: the first method to enable fully end-to-end learning of a 3D object detector from raw point clouds, eliminating all hand-crafted feature engineering that characterized earlier methods. Before VoxelNet, researchers manually designed features like LIDAR intensity histograms or geometric moment descriptors. VoxelNet replaces these with a learned Voxel Feature Encoding (VFE) layer.

The VFE process works as follows: the 3D point cloud is divided into a regular grid of small cubic voxels (e.g., $0.4\text{m} \times 0.4\text{m} \times 0.4\text{m}$). Within each non-empty voxel, a small number of points are sampled and passed through a shared MLP, then element-wise max-pooled into a single fixed-size feature vector. This transforms the irregular, unordered point cloud into a structured 3D feature tensor that conventional convolutional networks can process.

After VFE, a stack of 3D convolutional layers aggregates spatial context across the voxel grid, collapsing the depth dimension. The resulting 2D feature map is passed to a Region Proposal Network (RPN) — borrowed directly from 2D object detection — to predict class probabilities and 3D bounding box parameters. Despite its breakthrough status, VoxelNet's main limitation was slow inference (~ 4.4 Hz) due to dense 3D convolutions, which later motivated sparse convolution methods like SECOND and PointPillars.

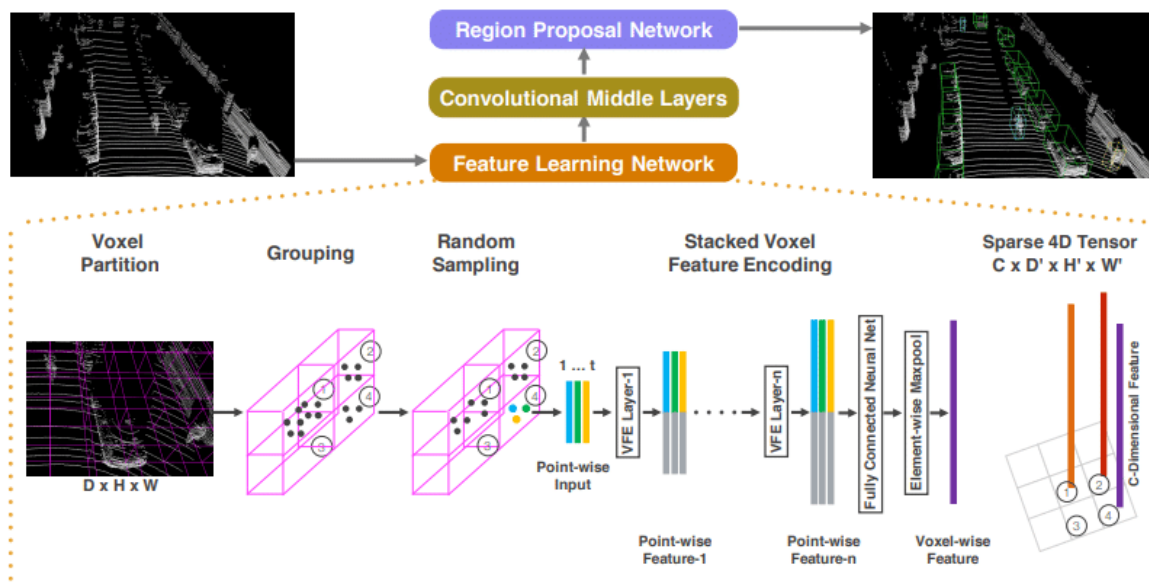
Datasets Used

- KITTI — outdoor driving scenes. Evaluated on the standard 3D detection benchmark for cars, pedestrians, and cyclists across Easy/Moderate/Hard difficulty tiers.

Architecture

```

Input Point Cloud (N ~ 100,000 points, xyz + reflectance)
|
v
Voxelization: divide space into D x H x W grid (e.g., 10 x 400 x 352)
--> Assign each point to its voxel cell
--> Sample T = 35 points per non-empty voxel
|
v
Voxel Feature Encoding (VFE) – 2 or 3 stacked layers:
For each voxel:
  - Per-point MLP (point features + voxel-mean offset)
  - Element-wise max-pool across points
  - Concatenate per-point and pooled features
--> Voxel feature vector (C dims per voxel)
|
v
Sparse 3D Voxel Feature Volume (C x D x H x W)
|
v
3D Convolutional Middle Layers
--> Downsample depth D, extract 3D spatial context
--> Reshape to 2D: (C' x H' x W')
|
v
Region Proposal Network (RPN)
--> 3 downsampling + 3 upsampling blocks
--> Output: objectness score + 3D box regression per anchor
  
```



Performance Results

Dataset	Class	Difficulty	3D AP
KITTI	Car	Easy	77.47
KITTI	Car	Moderate	65.11
KITTI	Car	Hard	57.73
KITTI	Pedestrian	Moderate	33.69

Open Problems

- Slow inference (~4.4 Hz) due to dense 3D convolutions applied over a large voxel grid, most of which is empty. This was the primary motivation for sparse convolution methods.
 - High memory footprint from the dense 3D feature volume, making it difficult to scale to larger detection ranges or finer voxel resolution.
 - The fixed voxel resolution introduces a quantization error — fine-grained geometric details smaller than the voxel size are permanently lost.
 - Performance on small objects (pedestrians, cyclists) is notably worse due to the sparse point coverage per voxel at long range.
-

4. PointPillars (CVPR 2019)

Paper: <https://arxiv.org/abs/1812.05784>

Code: <https://github.com/nutonomy/second.pytorch>

Citation: Lang, A. H., Vora, S., Caesar, H., Zhou, L., Yang, J., & Beijbom, O. (2019). PointPillars: Fast Encoders for Object Detection from Point Clouds. CVPR 2019.

Abstract

PointPillars is arguably the most impactful method in this survey from a production deployment perspective. The insight that drives the entire design is simple but powerful: for autonomous driving, objects of interest (cars, pedestrians, cyclists) are all on the ground plane — detecting them in bird's-eye view (BEV) is sufficient. This means the vertical (Z) dimension carries little discriminative information, and expensive 3D convolutions are unnecessary.

Instead of voxels, PointPillars partitions the point cloud into vertical columns called "pillars" — each pillar is a cell in an X-Y grid that extends infinitely in the Z direction. Within each pillar, points are encoded with augmented features (their position relative to the pillar center and to the mean of all pillar points), then passed through a small PointNet-style MLP and max-pooled into a single fixed-length feature vector per pillar. These pillar features are then scattered back onto a 2D grid, creating a "pseudo-image" where each pixel corresponds to one pillar.

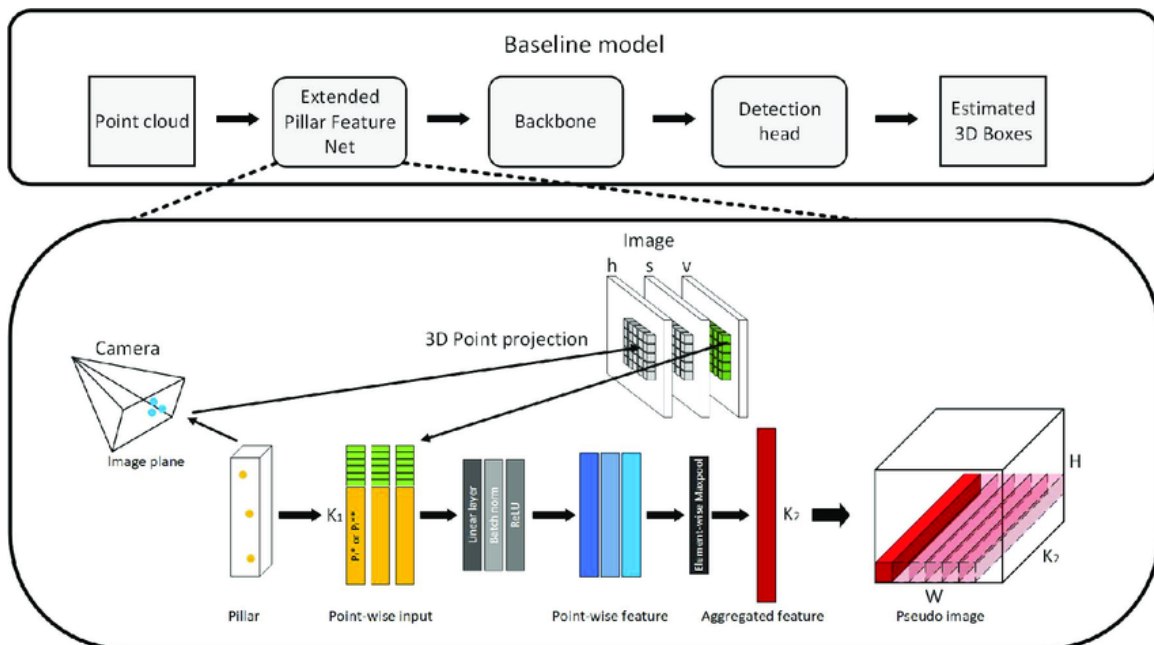
This pseudo-image is processed by a standard 2D CNN backbone (similar to those used for 2D image detection), completely avoiding 3D convolutions. The detection head uses an SSD-style (Single Shot Detector) approach to predict class labels and 3D box parameters for each anchor. The result is dramatic speed improvement: 62 Hz inference on a GPU, versus ~5 Hz for VoxelNet, while matching or exceeding accuracy. PointPillars became the standard backbone for real-time production deployment in autonomous vehicles.

Datasets Used

- KITTI — primary evaluation benchmark for outdoor driving (cars, pedestrians, cyclists).
- nuScenes — also widely evaluated on this multi-class urban driving dataset with 10 categories.
- Deployed in proprietary production AV stacks by multiple companies (nuTonomy, Aptiv, and many others).

Architecture

```
Input Point Cloud (N points, xyz + reflectance)
|
v
Pillar Partition: divide X-Y plane into H x W cells (e.g., 0.16m x 0.16m each)
--> Each pillar can hold up to P = 100 points
--> Points encoded with 9 features: [x, y, z, r, xc, yc, zc, xp, yp]
    (c = offset from pillar mean, p = offset from pillar center)
|
v
Pillar Feature Encoding (mini PointNet per pillar):
- Shared linear + BN + ReLU
- Element-wise max-pool across P points
--> One feature vector (C = 64 dims) per non-empty pillar
|
v
Scatter back to 2D pseudo-image (H x W x C)
|
v
2D CNN Backbone (3 downsampling blocks + 3 upsampling blocks with skip connections)
|
v
SSD-style Detection Head:
- Anchors at each spatial location (per class, per orientation)
- Predicts: class score + box (x, y, z, l, w, h, yaw)
```



Performance Results

Dataset	Class	Difficulty	3D AP	Speed
KITTI	Car	Easy	82.58	62 Hz
KITTI	Car	Moderate	74.31	62 Hz
KITTI	Car	Hard	68.99	62 Hz
KITTI	Pedestrian	Moderate	~52	62 Hz

Open Problems

- The BEV representation completely discards vertical structure — objects of similar footprint but different heights cannot be distinguished, and overhead obstacles are entirely missed.
 - Pillars are axis-aligned rectangles, so rotated or non-axis-aligned objects are not handled natively; the model must learn this through data augmentation.
 - Pedestrian detection accuracy lags significantly behind car detection due to the sparse point coverage of small, narrow objects at range.
 - Fixed pillar resolution creates a speed-accuracy tradeoff: finer pillars improve accuracy but slow inference; coarser pillars are faster but lose detail.
-

5. PointRCNN (CVPR 2019)

Paper: <https://arxiv.org/abs/1812.04244>

Code: <https://github.com/sshaoshuai/PointRCNN>

Citation: Shi, S., Wang, X., & Li, H. (2019). PointRCNN: 3D Object Proposal Generation and Detection from Point Cloud. CVPR 2019.

Abstract

PointRCNN is the point-cloud analog of Faster R-CNN: a two-stage detector that first generates coarse object proposals, then refines them into precise bounding boxes. Unlike PointPillars or VoxelNet which convert points into grid structures, PointRCNN works entirely on raw points throughout both stages.

In Stage 1, a PointNet++ backbone processes the full input point cloud and generates per-point feature vectors. A segmentation head uses these features to label every point as foreground (part of an object) or background. Each foreground point independently generates a 3D bounding box proposal using bin-based regression (x and z coordinates are divided into discrete bins, reducing the regression problem). After non-maximum suppression, the top-K proposals are kept.

Stage 2 crops the points inside each proposal, transforms them into a canonical coordinate frame centered on the proposal (making the problem rotation and translation invariant), and passes them through another PointNet++ to predict refined class scores and box corrections. This canonical refinement is key to PointRCNN's accuracy: by normalizing coordinates relative to each proposal, the network can focus on fine-grained shape details rather than absolute scene position. PointRCNN demonstrated that pure point-based detection could match voxel-based methods on KITTI without any discretization.

Datasets Used

- KITTI — primary benchmark. Evaluated on cars, pedestrians, and cyclists across Easy/Moderate/Hard tiers using the standard 3D AP metric.

Architecture

=== STAGE 1: Bottom-Up Proposal Generation ===

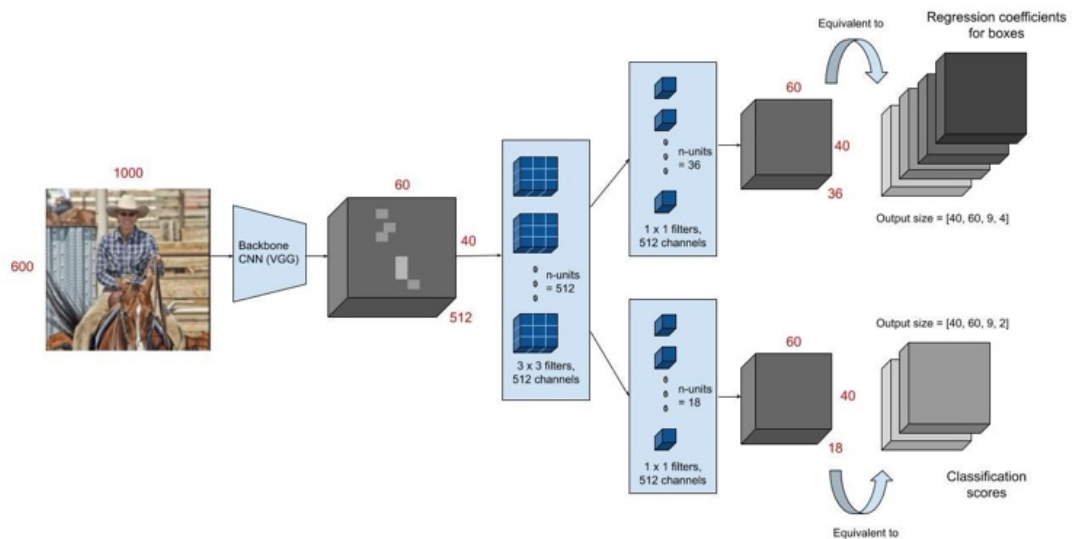
```

Input Point Cloud (N ~ 16,000 points, xyz + reflectance)
|
v
PointNet++ Backbone (SA + FP layers)
--> Per-point feature vectors (N x C)
|
+-----+
|               |
v               v
Segmentation Head   Proposal Head
foreground/background  bin-based 3D box per foreground point
|               |
+-----+-----+
|               |
v               v
NMS --> Top-K proposals (K = 512 in training)
  
```

=== STAGE 2: Canonical Box Refinement ===

```

For each proposal:
- Crop points within enlarged box region
- Transform to canonical coords (center at origin, heading = 0)
- Concatenate semantic features from Stage 1
|
v
PointNet++ feature extraction on cropped canonical points
|
v
Classification head + Box refinement head
--> Final 3D box (center, size, heading)
  
```



Performance Results

Dataset	Class	Difficulty	3D AP
KITTI	Car	Easy	85.94
KITTI	Car	Moderate	75.76
KITTI	Car	Hard	68.32
KITTI	Pedestrian	Moderate	41.78
KITTI	Cyclist	Moderate	58.82

Open Problems

- Two-stage inference is significantly slower than single-stage methods (~10 Hz vs. 62 Hz for PointPillars), limiting deployment to offline or near-real-time scenarios.
 - Foreground segmentation quality directly determines proposal recall — points from small or distant objects are often classified as background, making them undetectable in Stage 2.
 - The method is heavily tuned for outdoor KITTI driving scenarios; transferring to indoor scenes requires substantial adaptation of the proposal generation strategy.
 - Memory and compute costs are high due to the need to process all N points in Stage 1 and then separately process crops for each of the K proposals in Stage 2.
-

6. PV-RCNN (CVPR 2020)

Paper: <https://arxiv.org/abs/1912.13192>

Code: <https://github.com/open-mmlab/OpenPCDet>

Citation: Shi, S., Guo, C., Jiang, L., Wang, Z., Shi, J., Wang, X., & Li, H. (2020). PV-RCNN: Point-Voxel Feature Set Abstraction for 3D Object Detection. CVPR 2020.

Abstract

PV-RCNN (Point-Voxel Region CNN) is a hybrid architecture that deliberately combines both the voxel-based and point-based paradigms, arguing that the two are complementary rather than competing. Voxel-based methods (like VoxelNet) are excellent at capturing dense spatial structure efficiently via 3D convolutions, but they lose fine-grained point-level geometry during voxelization. Point-based methods (like PointRCNN) preserve fine geometry but are slower and scale poorly to large point clouds. PV-RCNN integrates both.

The architecture maintains two parallel branches. A Sparse 3D CNN operates on the voxelized point cloud and generates multi-scale voxel feature maps — these efficiently capture large-scale spatial structure. Simultaneously, a small set of $M = 2,048$ "keypoints" are sampled from the raw input using farthest point sampling. A novel Voxel-to-Keypoint Feature Aggregation (VSA) module then transfers features from the multi-scale voxel branches to these sparse keypoints, giving each keypoint access to rich spatial context.

In Stage 1, the voxel branch's RPN generates 3D proposals. In Stage 2, RoI-grid pooling samples keypoint features within and around each proposal box, creating a compact representation for final classification and box refinement. PV-RCNN achieved state-of-the-art results on both KITTI and the large-scale Waymo Open Dataset, demonstrating that hybrid representations consistently outperform single-paradigm methods at the cost of greater architectural complexity.

Datasets Used

- KITTI — primary evaluation benchmark. Achieves state-of-the-art at publication on car, pedestrian, and cyclist 3D AP.
- Waymo Open Dataset — large-scale outdoor dataset with ~1,000 driving scenes and 3D annotations for vehicles and pedestrians. Evaluated at LEVEL 1 and LEVEL 2 difficulty.

Architecture

Input Point Cloud (N points, xyz + reflectance)

Voxelization
 FPS Sampling
 --> M = 2,048 keypoints

Sparse 3D CNN
 (multi-scale feature maps)

Voxel-to-Keypoint Set Abstraction (VSA):

- For each keypoint, aggregate voxel features from multiple scales via ball query
- > Keypoint feature vectors (M x C)

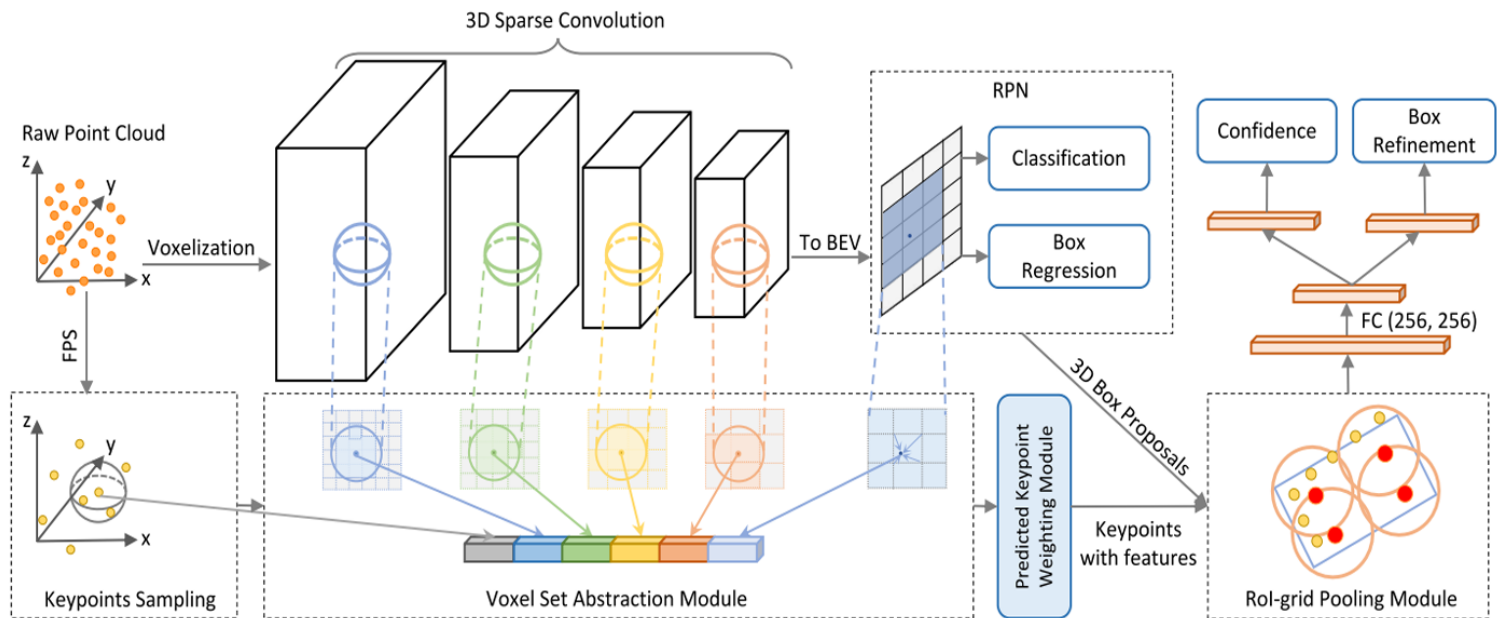
Stage 1: Voxel-branch RPN

- > 3D box proposals (from BEV voxel features)

Stage 2: RoI-Grid Pooling

- Sample $G \times G \times G$ grid points inside each proposal
- Aggregate keypoint features via set abstraction
- > Proposal feature vector

Refinement Head: Classification + Box correction



Performance Results

Dataset	Class	Difficulty	3D AP
KITTI	Car	Easy	90.25
KITTI	Car	Moderate	81.43
KITTI	Car	Hard	76.82
Waymo Open	Vehicle	LEVEL 1	77.51
Waymo Open	Pedestrian	LEVEL 1	~51

Open Problems

- Architectural complexity is very high — two parallel feature extraction branches plus cross-branch aggregation make the model difficult to tune and maintain.
 - Inference speed is moderate (~12 Hz) and significantly slower than single-stage methods, limiting suitability for real-time deployment without further optimization.
 - The VSA module introduces a large number of hyperparameters (ball query radii, number of keypoints, grid sampling density), each requiring careful tuning per dataset.
 - Computational cost scales poorly with scene size: larger detection ranges require more keypoints and wider voxel grids, inflating memory and compute requirements.
-

7. CenterPoint (CVPR 2021)

Paper: <https://arxiv.org/abs/2006.11275>

Code: <https://github.com/tianweiy/CenterPoint>

Citation: Yin, T., Zhou, X., & Krähenbühl, P. (2021). Center-based 3D Object Detection and Tracking. CVPR 2021.

Abstract

CenterPoint fundamentally reimagines how 3D bounding boxes are predicted. Traditional methods use anchors — pre-defined boxes at fixed orientations and sizes tiled across the detection space. The model then predicts corrections to these anchors. This is problematic in 3D because objects can have arbitrary orientations, requiring many anchors per location to cover all possible headings, which creates redundancy and increases NMS complexity.

CenterPoint instead predicts object centers as 2D heatmap peaks in the bird's-eye view feature map — one peak per object, regardless of its heading. This is directly inspired by CenterNet, a 2D keypoint detector. From each detected center, a set of lightweight regression heads then predict all other box attributes: the height above ground (Z), the 3D box dimensions (length, width, height), the heading angle (via sin/cos representation), velocity (for tracking), and per-class confidence.

Because heading angle is decoupled from the proposal stage, the model has far fewer proposals to evaluate and no NMS is needed over orientation hypotheses. An optional Stage 2 point-cloud refinement module can further improve box accuracy. CenterPoint also produces object velocity estimates, enabling dead-reckoning-based multi-object tracking essentially for free — the model outputs trackable detections without a separate tracker. It achieved state-of-the-art on both nuScenes and Waymo Open at publication and remains a widely used baseline today.

Datasets Used

- nuScenes — large-scale urban autonomous driving dataset with 1,000 driving scenes, 10 object classes, annotated at 2 Hz with LIDAR, camera, and radar. Primary metric: NDS (nuScenes Detection Score) and mAP.
- Waymo Open Dataset — evaluated on vehicle and pedestrian classes at LEVEL 1 and LEVEL 2 difficulty.

Architecture

```

Input Point Cloud
|
v
Voxel Encoder (PointPillars or VoxelNet-style)
--> encode raw points into voxel/pillar features
|
v
Sparse 3D CNN Backbone
--> multi-scale spatial feature extraction
|
v
BEV Feature Map (H x W x C) – top-down view of scene
|
v
Center Heatmap Head (one heatmap per class):
- Gaussian peaks at true object centers
- Peak detection replaces NMS over anchors
--> Set of (x, y) object center locations
|
v
Per-Center Regression Heads (evaluated at each peak):
- Z (height above ground)
- 3D size (L, W, H)
- Heading (sin  $\theta$ , cos  $\theta$ )
- Velocity (vx, vy) for tracking
- Per-class confidence
|
v
[Optional] Stage 2 Point-Level Refinement
|
v
Final 3D Detections + Tracking IDs

```

Performance Results

Dataset	Metric	Score
nuScenes	NDS (nuScenes Detection Score)	65.5
nuScenes	mAP	58.0
Waymo Open	3D AP (Vehicle, LEVEL 1)	80.2
Waymo Open	3D AP (Pedestrian, LEVEL 1)	78.3

Open Problems

- The BEV representation discards all vertical (Z-axis) structure, making it impossible to distinguish objects stacked vertically (e.g., vehicles on a transporter) or overhead obstacles.
 - Center heatmaps can struggle to separate closely spaced objects of the same class — two nearby cars may produce a single merged peak rather than two distinct centers.
 - Performance on irregular or articulated objects (e.g., buses, construction vehicles) is limited because the model assumes compact bounding boxes.
 - Still dependent on voxel encoders, so resolution and range are constrained by the underlying voxelization step.
-

8. ImVoteNet (CVPR 2020)

Paper: <https://arxiv.org/abs/2001.10692>

Code: <https://github.com/facebookresearch/invotenet>

Citation: Qi, C. R., Chen, X., Litany, O., & Guibas, L. J. (2020). ImVoteNet: Boosting 3D Object Detection in Point Clouds with Image Votes. CVPR 2020.

Abstract

ImVoteNet is a direct extension of VoteNet that incorporates RGB image information into the 3D detection pipeline. Its starting motivation is straightforward: RGB-D sensors (like the Microsoft Kinect or Intel RealSense used to capture indoor datasets) simultaneously provide both a depth point cloud and a color image. VoteNet uses only the 3D geometry; ImVoteNet asks — what if we also used the image?

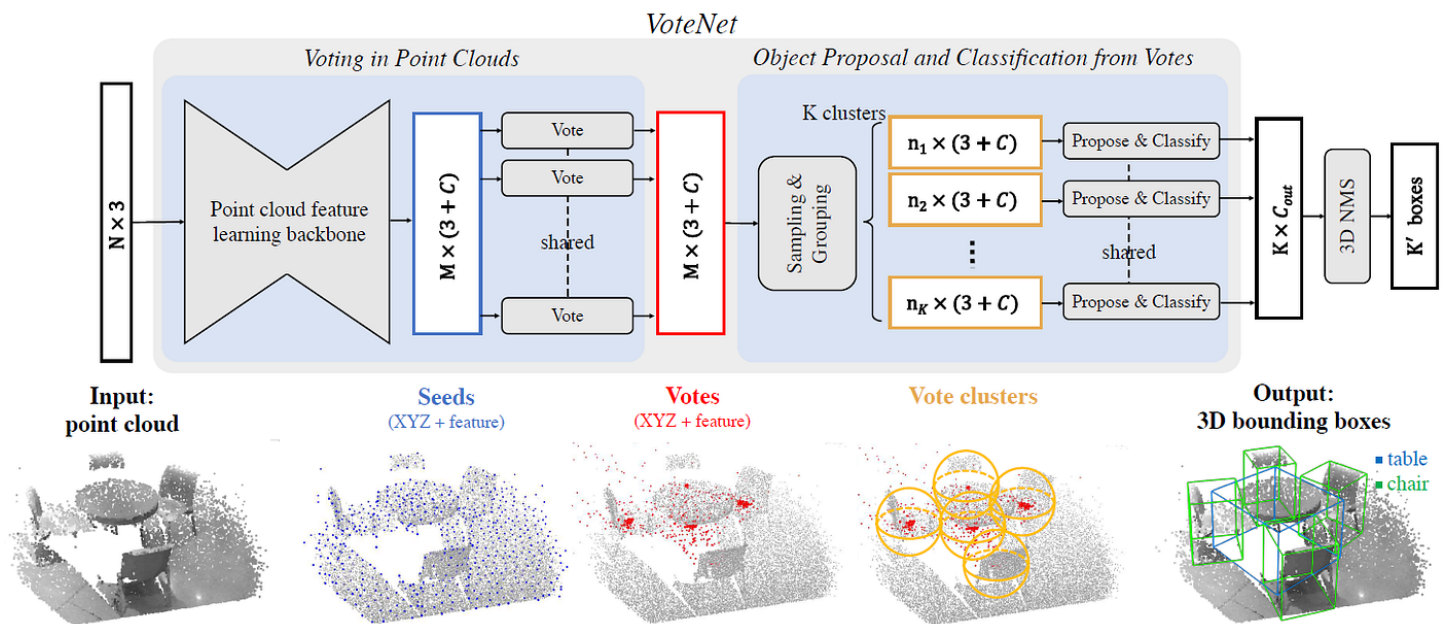
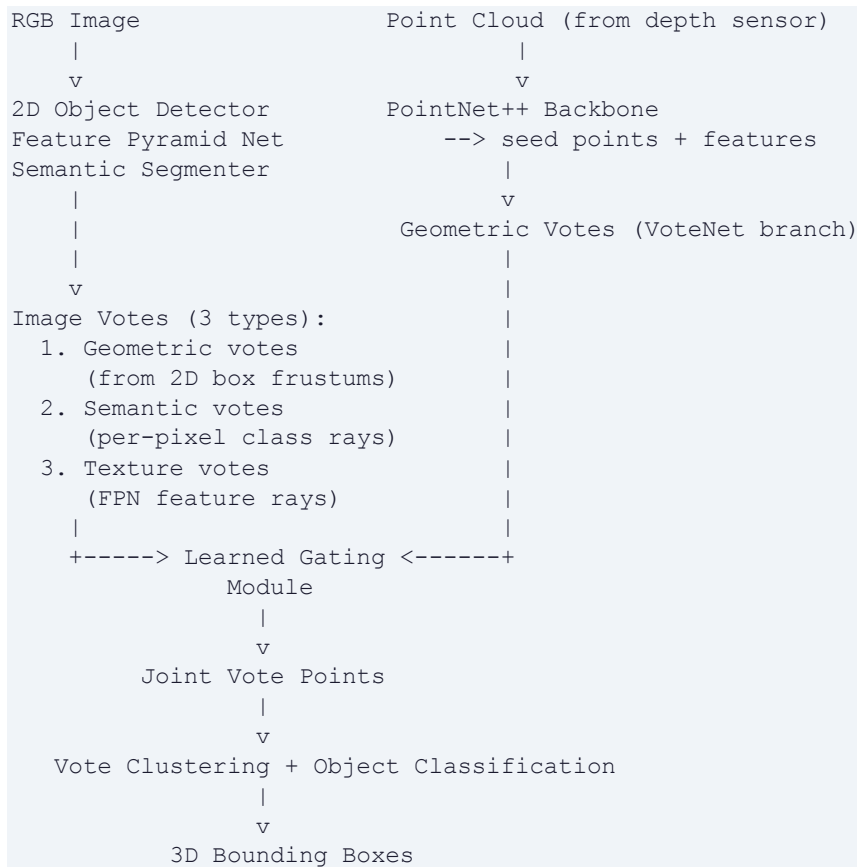
The key challenge in combining 2D and 3D is alignment: a 2D pixel corresponds to a ray in 3D space, not a specific point. ImVoteNet introduces three types of image-derived votes, each exploiting a different aspect of image information. Geometric votes extrude 2D object detections into 3D frustums, producing votes from image-detected object locations. Semantic votes use per-pixel class predictions to lift pixel-level semantic labels into 3D rays. Texture votes use image feature descriptors (from a feature pyramid) pooled along projection rays to produce texture-aware 3D signals.

These three types of image votes are fused with the geometric votes produced by the original VoteNet backbone via a learned gating module that weights the contributions of image and geometry cues based on confidence and compatibility. The fused votes are then clustered and classified exactly as in VoteNet. On SUN RGB-D, ImVoteNet improves mAP@0.25 by 5.7 points over VoteNet — a substantial margin for indoor detection.

Datasets Used

- SUN RGB-D — primary evaluation. 10,355 RGB-D images from four different sensor types (Kinect v1/v2, RealSense, Structure Sensor) covering 10 indoor object classes. Both RGB and depth are used.
- ScanNet V2 — also evaluated, where the RGB frames are used alongside the 3D reconstructed point cloud.

Architecture



Performance Results

Dataset	Metric	Score	vs. VoteNet
SUN RGB-D	mAP @ IoU 0.25	63.4	+5.7
SUN RGB-D	mAP @ IoU 0.50	~37	+~3.5
ScanNet V2	mAP @ IoU 0.25	~61	+~2.4

Open Problems

- The method requires tightly aligned RGB-D data — pure LIDAR scenes without color information receive no benefit from the image branch.
 - The image branch (2D detector + FPN + per-pixel segmentation) adds significant compute overhead, making ImVoteNet substantially heavier than VoteNet alone.
 - Projection-based fusion introduces geometric ambiguity — a 2D pixel maps to a ray in 3D, and vote positions along that ray must be inferred, introducing depth uncertainty.
 - The gating module may fail in low-light or texture-poor environments where image features are uninformative, potentially degrading below VoteNet baseline.
-

9. 3DETR (ICCV 2021)

Paper: <https://arxiv.org/abs/2109.08141>

Code: <https://github.com/facebookresearch/3detr>

Citation: Misra, I., Girdhar, R., & Joulin, A. (2021). *An End-to-End Transformer Model for 3D Object Detection*. ICCV 2021.

Abstract

3DETR (3D DEtection TRansformer) brings the DETR paradigm — originally introduced for 2D image object detection — to 3D point clouds. DETR replaced the traditional anchor + NMS pipeline in 2D detection with a Transformer encoder-decoder that uses learnable object queries to directly predict a set of bounding boxes. 3DETR does the same for 3D.

The architecture uses PointNet++-style set abstraction layers as a backbone to downsample the raw point cloud into $M = 2,048$ seed points. These seed points are encoded by a Transformer encoder using self-attention, where every point can directly attend to every other point to build a global scene representation. A Transformer decoder then takes $K = 256$ learnable object queries (random or FPS-initialized 3D position embeddings) and uses cross-attention to bind each query to the relevant region of the encoded scene features.

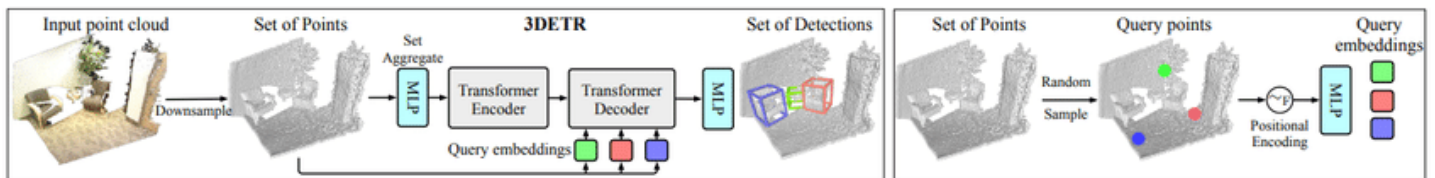
This process is entirely end-to-end and eliminates several hand-engineered components: no voting, no Euclidean-distance clustering, no non-maximum suppression, and no learned anchors. The Hungarian matching algorithm is used during training to assign queries to ground-truth objects. 3DETR demonstrates that Transformers can replace the complex, carefully engineered components of VoteNet-style detectors with a single, simpler architecture. A pre-trained variant, 3DETR-m, using masked encoder pretraining, further improves results and showed the power of self-supervised pretraining for 3D understanding.

Datasets Used

- ScanNet V2 — evaluated on the standard indoor 3D detection benchmark (18 classes, mAP @ IoU 0.25 and 0.50).
- SUN RGB-D — evaluated on the 10-class indoor RGB-D benchmark.

Architecture

```
Input Point Cloud (N = 40,000 points, xyz + optional color)
|
v
Set Abstraction Layers (PointNet++ style):
- FPS + ball query grouping
- Hierarchical downsampling
--> M = 2,048 seed points + features
|
v
Transformer ENCODER:
- Self-attention: every seed attends to every other seed
- Sinusoidal 3D positional encoding from xyz coords
- L encoder layers (L = 3)
--> Encoded scene feature set (M x d_model)
|
v
Transformer DECODER:
- K = 256 learnable object queries
- Each query = position embedding (random or FPS init)
- Cross-attention: queries attend to encoded scene
- Self-attention: queries attend to each other
- L decoder layers (L = 8)
--> Decoded query features (K x d_model)
|
v
MLP Prediction Heads (per query):
- Class probability (softmax over C+1 classes + no-object)
- 3D center (xyz)
- Box dimensions (l, w, h)
- Objectness confidence
|
v
Final detections (Hungarian matching during training, threshold at inference)
```



Performance Results

Dataset	Variant	mAP @ 0.25	mAP @ 0.50
ScanNet V2	3DETR	62.7	37.5
ScanNet V2	3DETR-m (pretrained)	65.0	47.0
SUN RGB-D	3DETR	58.0	30.3
SUN RGB-D	3DETR-m (pretrained)	59.1	32.7

Open Problems

- Self-attention in the encoder scales quadratically with the number of seed points ($O(M^2)$), making it computationally expensive at fine resolutions.
 - Transformer training is slow to converge and typically requires more epochs and more careful learning rate scheduling than CNN-based methods.
 - Positional encoding for 3D points is less well-understood than for 1D sequences or 2D images — different encoding strategies significantly affect performance.
 - The model is less interpretable than VoteNet: it is difficult to visualize which scene points each query is attending to or why a particular object was detected.
-

10. GroupFree-3D (ICCV 2021)

Paper: <https://arxiv.org/abs/2104.00678>

Code: <https://github.com/zeliu98/Group-Free-3D>

Citation: Liu, Z., Zhang, Z., Cao, Y., Hu, H., & Tong, X. (2021). Group-Free 3D Object Detection via Transformers. ICCV 2021.

Abstract

GroupFree-3D identifies a fundamental bottleneck in VoteNet-style detectors: the grouping step, where votes are clustered into object proposals using a fixed Euclidean distance radius. This hard spatial grouping makes two critical assumptions — that an object's votes will be within a fixed distance of each other, and that points from different objects will not intermix within that radius. Both assumptions frequently fail in cluttered indoor scenes with densely packed furniture.

The solution is to eliminate explicit grouping entirely by using a Transformer decoder where each candidate object proposal can freely attend to all seed points in the scene via cross-attention — not just those within a hand-tuned neighborhood radius. The model itself learns which points are relevant to which object through the attention mechanism, making the grouping soft, global, and fully data-driven.

The architecture begins with a PointNet++ backbone (shared with VoteNet) to extract seed point features. A set of K initial candidate object locations are selected (either by confidence or FPS). These candidates act as queries in a multi-layer Transformer decoder, iteratively refining their features and predicted box parameters through repeated cross-attention to all seed points and self-attention among candidates. Each decoder layer produces a prediction, and an auxiliary loss is applied at every layer for stable training. GroupFree-3D achieved the best indoor 3D detection results of 2021, outperforming both VoteNet and 3DETR on ScanNet V2 and SUN RGB-D.

Datasets Used

- ScanNet V2 — primary benchmark. Achieves 69.1 mAP@0.25 and 52.8 mAP@0.50 — best results on ScanNet V2 in 2021.
- SUN RGB-D — achieves 63.0 mAP@0.25 and 45.2 mAP@0.50, also best results in 2021.

Architecture

```
Input Point Cloud (N points, xyz + optional color/normals)
|
v
PointNet++ Backbone (SA layers):
--> M = 1,024 seed points + feature vectors
|
v
Initial Candidate Selection:
- K = 256 candidates (top-K by objectness or FPS)
- Each candidate assigned initial position from seed points
|
v
Multi-Layer Transformer DECODER (L layers, e.g. L = 6 or 12):
  Each layer performs:
    (a) Self-Attention among K candidates
        --> candidates share context with each other
    (b) Cross-Attention: each candidate attends to ALL M seeds
        --> no locality constraint - "group-free"
        --> attention weights learned, not proximity-based
    (c) FFN (feed-forward network)
  + Auxiliary detection loss at EVERY layer
  |
  v
Per-Layer & Final Prediction Heads (per candidate):
- Objectness score
- Class probabilities
- 3D center (xyz)
- Box dimensions (l, w, h)
- Heading angle
|
v
Final 3D Detections (from last decoder layer)
```

Performance Results

Dataset	mAP @ IoU 0.25	mAP @ IoU 0.50
ScanNet V2	69.1	52.8
SUN RGB-D	63.0	45.2
ScanNet V2 (VoteNet, for comparison)	58.6	33.5
SUN RGB-D (VoteNet, for comparison)	57.7	—

Open Problems

- Global cross-attention over all M seed points for all K candidates is computationally heavy — training and inference are significantly slower than VoteNet.
 - Requires more training epochs to converge than VoteNet, increasing the computational cost of experimentation and hyperparameter tuning.
 - Interpretability is poor: the learned attention patterns determine grouping but are high-dimensional and difficult to visualize or debug.
 - The iterative refinement through decoder layers gives the model more capacity but also more opportunity for instability — auxiliary losses at each layer are necessary to stabilize training.
-

